

The University of Akron

IdeaExchange@UAkron

Williams Honors College, Honors Research
Projects

The Dr. Gary B. and Pamela S. Williams Honors
College

Spring 2026

Security Onion IDS Case Study: Detecting and Investigating Threat Traffic Using Suricata, Zeek, and PCAP Evidence

Daryna Myroniuk

The University of Akron, dm421@uakron.edu

Follow this and additional works at: https://ideaexchange.uakron.edu/honors_research_projects



Part of the [Other Computer Engineering Commons](#)

Please take a moment to share how this work helps you [through this survey](#). Your feedback will be important as we plan further development of our repository.

Recommended Citation

Myroniuk, Daryna, "Security Onion IDS Case Study: Detecting and Investigating Threat Traffic Using Suricata, Zeek, and PCAP Evidence" (2026). *Williams Honors College, Honors Research Projects*. 2214.

https://ideaexchange.uakron.edu/honors_research_projects/2214

This Dissertation/Thesis is brought to you for free and open access by The Dr. Gary B. and Pamela S. Williams Honors College at IdeaExchange@UAkron, the institutional repository of The University of Akron in Akron, Ohio, USA. It has been accepted for inclusion in Williams Honors College, Honors Research Projects by an authorized administrator of IdeaExchange@UAkron. For more information, please contact cjg74@uakron.edu.

**Security Onion IDS Case Study: Detecting and Investigating Threat Traffic Using Suricata,
Zeek, and PCAP Evidence**

Daryna Myroniuk

Department of CIS - Cybersecurity

Honors Research Project

Submitted to

*The Williams Honors College
The University of Akron*

Approved:

Nadhem Ebrahim

Honors Project Sponsor (signed)

Nadhem Ebrahim 4/23/2026

Honors Project Sponsor (printed) Date

Richard P Mehok Jr.

Honors Project Reader (signed)

Richard P Mehok Jr. 4-23-26

Honors Project Reader (printed) Date

Stanley Smith

Honors Project Reader (signed)

Stanley Smith 4/22/2026

Honors Project Reader (printed) Date

Accepted:

Janet Kropff

Honors Faculty Advisor (signed)

Janet Kropff 4/24/26

Honors Faculty Advisor (printed) Date

Tim O'Neil

Department Chair (signed)

Tim O'Neil 5/4/2026

Department Chair (printed) Date



CIS Senior Cybersecurity Project to fulfill partial requirements of the Bachelor of
Computer Information Systems-Cybersecurity

**Security Onion IDS Case Study: Detecting and Investigating Threat Traffic Using
Suricata, Zeek, and PCAP Evidence**

By

Student Name: - Daryna Myroniuk

Section number: 001

Supervisor: Nadhem Ebrahim

Date: 4/15/2026

Abstract

A key component of cybersecurity is network intrusion detection, which is used to examine network traffic for any malicious activity. Many organizations deploy intrusion detection systems (IDS) but still face challenges such as validating detections, investigating alerts in a timely manner, and producing clear and repeatable evidence of what has occurred, especially when live traffic capture may be limited to risks, permissions, or privacy concerns. The goal of this project was to use Security Onion, which includes Suricata and Zeek to create and demonstrate a controllable and manageable evidence-based IDS investigation workflow.

This project is focused on deploying and validating a Security Onion sensor in a live enterprise environment to support passive network monitoring and evidence-based investigations. The main goal was to use Suricata and Zeek tools built into the Security Onion's system, as well as packet capture features to build a practical workflow for detecting and investigating suspicious traffic. The project was centered on the sensor side of the platform rather than full manager deployment. The sensor was configured as a virtual machine with separate management and monitoring interfaces so it could receive mirrored traffic without interrupting production systems.

The work approach included preparing the virtual sensor environment, verifying interface roles, confirming that the monitoring interface could receive traffic, enabling and tuning Suricata and Zeek, adjusting HOME_NET for the monitored subnet, reviewing retention and storage settings, and validating that alerts, logs, and packet evidence could be produced and examined. Troubleshooting was also a major part of the project, especially when confirming where Suricata event files were written, how data appeared in Hunt, and how alert results should be filtered and interpreted.

The project succeeded in creating a functional Security Onion sensor workflow that was able to process mirrored traffic and generate useful investigation data through Suricata alerts, Zeek logs, and packet evidence. The project proves that a properly deployed virtual Security Onion sensor can improve network visibility and provide a practical, low-cost method for supporting intrusion detection investigations in an enterprise environment. Future improvements could expand traffic coverage and long-term alert retention and evidence management.

Table of Contents

Contents

Abstract	II
Table of Contents	IV
Chapter 1: Introduction	1
Overview	1
Motivation	2
Research Objectives	3
Scope and limitations	5
Contribution	6
Societal impacts	7
Ethics	8
Risk Management	9
Chapter 2: Background and Related Work	11
Overview	11
Framework and Threat Model Alignment	12
Existing systems or approaches	12
Limitations of current systems	14
Conclusion	15
Chapter 3: Problem statement and system design	17
3.1 Overview and Problem Statement	17
3.2 Methodologies	19
3.2.1 Initial Planning and Design Approach	20
3.2.2 Sensor Preparation and Platform Validation	21
3.2.3 Interface Identification and Monitoring Readiness	22
3.2.4 Configuration and Internal Tuning	24
3.2.5 Alert, Log, and Workflow Validation	26
3.3 System design	28
3.3.1 Overall Architecture	28
3.3.2 Sensor Role and Interface Design	29
3.3.3 Sensor Deployment and Monitoring Path	31
3.3.4 SPAN Validation and Investigation Workflow	32
3.3.5 Repeatable Investigation Workflow	33

Chapter 4: Results and Analysis	34
4.1 Outcomes overview.....	34
4.2 Experimental results.....	34
4.2.1 Example Investigation of a Collected Alert	35
4.3 Analysis of the Results	37
4.4 Comparison with existing solutions	38
Conclusions and Future Work.....	40
Summary of Achievements	40
Limitations of the Project.....	40
Possible Future Improvements or Extensions.....	41
References.....	42
Appendices.....	44
Appendix A. Security Onion Service and Node Validation.....	44
Appendix B. Interface and Monitoring Path Validation.....	48
Appendix C. Traffic Capture Validation.....	50
Appendix D. Configuration and Tuning Evidence.....	52
Appendix E. Hunt, Alert, and Investigation Evidence	57

Chapter 1: Introduction

Overview

Network security is one of the most important parts of cybersecurity because organizations depend on their networks every day to support communication, services, and access to important systems. Suspicious activity can be missed, alerts can be hard to validate, and investigators may not have enough evidence to explain what happened clearly when network traffic is not monitored correctly. This becomes an even bigger problem in real enterprise environments, where privacy concerns are higher, access is restricted, and production risks can make live traffic monitoring and documentation more difficult.

This project focuses on improving that visibility by using Security Onion as a passive network intrusion detection platform. The main idea is to deploy and validate a virtual Security Onion sensor that can receive mirrored traffic from the network, inspect it without interrupting production systems, and turn that traffic into useful security evidence. The project uses Suricata to generate intrusion detection alerts, Zeek is used to record network and protocol metadata, and packet capture is used to preserve supporting evidence for deeper review. Instead of building a full Security Onion manager deployment, the focus of this work is on the sensor side of the process and on showing how a properly prepared sensor can support detection and investigation in a practical enterprise setting.

The purpose of this project is not only to show that the sensor can be deployed, but also to show that it can support a repeatable investigation workflow. That includes receiving approved mirrored traffic, producing alerts and logs, and helping explain suspicious events through a combination of Suricata, Zeek, and packet evidence. Overall, this project addresses the problem of limited network visibility by testing a realistic, low-cost, evidence-based

monitoring approach that can strengthen threat detection and make investigations easier to understand and document.

Motivation

This topic was chosen because network visibility is still one of the most important parts of security in real environments. A company can have endpoint tools and other protections in place, but that still cannot always provide a clear view of what is happening on the network itself or make an alert easy to validate. In intrusion detection, one of the biggest problems is not just getting alerts but being able to connect those alerts to useful evidence and then turn them into a clear investigation story. That matters even more in enterprise environments, where access is limited, network traffic is sensitive, and any change must be planned carefully to avoid affecting production systems. My project was motivated by this problem because even though our environment already had Security Onion setup it did not yet have the network sensor features needed to use Suricata, Zeek, and full packet capture.

This topic is also important because many real cyber threats can be seen in network traffic before the full story is visible from endpoint data alone. Monitoring network traffic helps security teams look for signs of initial access, command-and-control activity, data exfiltration, and lateral movement inside the environment. Those are all common parts of real incident investigations, which is why stronger network visibility is so important. In many environments, the challenge is not only detecting suspicious activity but also having enough context to understand what happened and explain it clearly.

Another reason I chose this project is because it gave me the opportunity to work on a real security problem instead of only using a classroom lab setup. Rather than just learning what an IDS does in theory, I was able to help prepare and validate a live Security Onion sensor

deployment in an enterprise environment. That made the project more meaningful because it involved real planning, troubleshooting, restricted access, and coordination with other IT staff. It also matched my goal of building something practical that improves visibility and supports investigations with alerts, logs, and packet evidence.

Research Objectives

Prepare and validate the Security Onion sensor environment:

- Deploy and prepare a virtual Security Onion sensor in the enterprise environment using separate management and monitoring interfaces.
- Review and adjust virtual machine resources, storage planning, and interface assignments so the sensor can operate reliably in a live enterprise setting.
- Confirm that the sensor is healthy and that core monitoring services such as Suricata, Zeek, and packet capture are running correctly before deeper validation begins.

Configure the sensor for correct monitoring and data handling:

- Identify and verify the correct management and monitoring interfaces, including how the monitoring interface is mapped inside the virtual sensor.
- Test and confirm that the monitoring interface can capture traffic correctly during the early validation stage before the final SPAN deployment is completed.
- Configure the monitored subnet correctly, including HOME_NET and related settings, so alerts and logs are labeled properly and are easier to interpret during analysis.
- Verify time, time zone, and synchronization settings so event timestamps remain accurate and investigation records stay consistent.

Validate detection, logging, and troubleshooting workflow:

- Confirm that Security Onion can generate usable Suricata alerts and Zeek logs during testing and troubleshooting.
- Troubleshoot issues related to Suricata output, including log location, EVE file generation, rotation, and event visibility inside the platform.
- Review how alerts are displayed in Hunt and Alerts so Suricata events, endpoint events, and other data sources can be separated and interpreted correctly.

Support practical monitoring and investigation:

- Evaluate retention, disk usage, and data management settings so alerts, logs, and packet evidence can be kept in a practical way without exhausting storage.
- Reduce unnecessary system load in a low-risk way by reviewing PCAP retention, limiting unneeded Zeek logs, and planning alert tuning instead of applying aggressive packet filtering too early.
- Coordinate the final design around approved SPAN or mirrored traffic requirements so the deployment fits enterprise restrictions and monitoring needs.
- Connect the sensor to approved mirrored traffic and confirm that monitored traffic reaches the correct monitoring interface after SPAN deployment.
- Build a repeatable investigation workflow that uses alerts, metadata, and packet evidence together to explain suspicious activity clearly and consistently.
- Document the final workflow, troubleshooting process, configuration choices, and results with written explanation and redacted screenshots suitable for the final report.

Scope and limitations

This project covers the deployment, preparation, and validation of a virtual Security Onion sensor used for passive network monitoring in a live enterprise environment. The main focus is on the sensor side of the monitoring process, not on building a full Security Onion manager from scratch. The project includes configuring the sensor, separating management and monitoring interfaces, also confirming that the correct sniffing interface is used, validating that mirrored traffic can reach the sensor, and confirming that Security Onion can turn monitored traffic into useful alerts, logs, and packet evidence for investigation. The project also covers the creation of a repeatable investigation workflow using Suricata alerts, Zeek logs, and PCAP evidence.

A real enterprise network was used for this project with the Security Onion sensor deployed as a virtual machine in vSphere. The sensor used separate interfaces for management and monitoring, with ens192 used for management and bond0 which is fed by ens224, used for traffic monitoring. The project worked with approved traffic only and was designed around a passive mirrored traffic feed through SPAN so that production traffic would not be interrupted. Early work was completed in a test and preparation stage before the final SPAN deployment was fully in place. Supporting interface and monitoring path evidence is provided in **Appendix B**.

The main tools used in this project were Security Onion, Suricata, Zeek, packet capture, Hunt, the Alerts view, SSH access, tcpdump, and system commands such as so-status, ip a, ip link, timedatectl, and bond interface checks. The systems analyzed were mainly the Security Onion sensor itself, its monitoring interfaces, the monitored server and infrastructure VLAN, as well as the security data produced by Suricata and Zeek. During testing and troubleshooting, the project examined Zeek connection and DNS visibility, Suricata EVE output, live alert filtering,

and packet capture behavior. Additional service validation, traffic capture, and configuration evidence are included in **Appendices A, C, and D**.

The project did not test live destructive attacks against production systems, and it was limited by enterprise privacy, access, and documentation restrictions. Because the work was done in a live monitored environment, many infrastructure views and implementation screens could not be captured directly. The project also did not aim to replace existing security tools, inspect unapproved traffic, or deploy inline prevention that could affect production operations. Storage and retention limits also affected how long logs and packet evidence could be kept on the system. This project was not supported by a complete step-by-step guide or prior documentation that could be found online. The design, troubleshooting, and validation process had to be worked out independently, which added to the overall challenge of the project.

Contribution

This project contributes a practical example of how Security Onion can be expanded from an Elastic-only setup into a more complete network monitoring workflow using a virtual sensor, mirrored traffic, and passive analysis. The main value of the project is not just the deployment itself, but the proof that a sensor-based approach can improve network visibility and support investigations in a real enterprise environment. It also adds value by showing the steps, troubleshooting, and design decisions needed to make that kind of deployment work under real access and privacy restrictions.

A second contribution to this project is that it turns the deployment into a usable investigation process rather than only a technical setup. By combining alerts, network metadata, and packet evidence, the project shows a clear way to move from suspicious traffic to a better explanation of what happened. This is useful because many environments already collect

security data, but not all of them have a simple and repeatable way to connect that data into a usable investigation narrative.

Societal impacts

This project has a positive societal impact because it supports better threat detection and faster investigation in environments that depend on secure network operations. Many organizations, including schools, public services, businesses, and local government systems, rely on network security to protect their systems and the people who use them. A monitoring approach that improves visibility can help reduce the chance that suspicious activity goes unnoticed. In that sense, projects like this can support stronger protection of important services, user data, and daily operations.

Another positive impact is that this project shows how a lower to no cost security platform can still provide meaningful results when it is deployed correctly. Not every organization can afford expensive commercial tools. Using Security Onion effectively can help smaller organizations improve their security without needing a large budget. This is important because stronger security should not only be available to large organizations with more resources. A project like this can help show that useful monitoring and investigation workflows can still be built with open and accessible tools.

At the same time, there are also possible negative impacts if a system like this is used improperly. Network monitoring tools can capture sensitive traffic and reveal private information if access is not controlled properly. They can also create concerns about privacy if people do not understand what is being monitored and why. In addition, if alerts are not interpreted correctly, security teams could waste time on false positives or misunderstand normal traffic as malicious

activity. Because of that, tools like this must be used responsibly, with clear limits, proper access control, and careful handling of evidence and screenshots.

Overall, the broader impact of this project is that it supports a more careful and evidence-based approach to cybersecurity. It can help organizations improve visibility, respond to incidents more clearly, and make better security decisions, while also showing the importance of privacy, responsible monitoring, and controlled use of security data.

Ethics

Ethical practice was an important part of this project because it was completed in a real enterprise environment with live network traffic and restricted access. The project was designed to use approved monitoring methods and only approved traffic sources. The sensor was configured for passive monitoring, which means it observed mirrored traffic without interrupting or changing normal production communication. This helped reduce risk to users and systems while still allowing the project to collect useful security data.

Another ethical concern in this project was privacy. Because network monitoring tools can capture sensitive technical details, I had to be careful about what information was viewed, stored, and documented. Any screenshots or report material had to be redacted so internal addresses, names, and other protected details were not exposed. In some cases, screenshots of certain interfaces were not included at all because even redacted versions could still reveal too much about the environment. That decision was important for protecting the organization while still documenting the work in a professional way.

This project also followed ethical standards by staying within the limits of my role and permissions. Since I was working as an intern, changes could not be made freely and had to be planned, reviewed, and completed in a controlled way. That meant the project was not only

about technical work, but also about acting responsibly, respecting access boundaries, and making sure the monitoring process did not create unnecessary risk. Overall, the ethical goal of this project was to improve security visibility in a way that was legal, careful, and respectful of privacy, system stability, and organizational trust.

Risk Management

This project involved several risks that had to be considered before and during the work. One major risk was accidental exposure of sensitive enterprise information, since network monitoring can capture internal traffic details, system activity, and other protected data. Another risk was the possibility of affecting production systems if the sensor was configured incorrectly or if the monitoring design was not kept passive. There were also project risks related to limited access, approval delays, incomplete traffic visibility, and storage limits, all of which could affect the quality of the results or slow down progress.

To manage these risks, the project was designed around approved monitoring only and a passive sensor model. The sensor received mirrored traffic instead of being placed inline, which reduced the chance of interfering with normal communication. Access was also controlled carefully because the work was done in a real enterprise environment and changes had to be planned and reviewed before being carried out. Sensitive details in screenshots and written documentation were redacted or left out when necessary to avoid exposing internal information. This helped protect the environment while still allowing the project to be documented.

Technical risks were also managed through step-by-step validation and troubleshooting. The sensor health, interface roles, traffic capture, alert generation, and log visibility were checked in stages instead of assuming everything was working from the beginning. Storage and retention were reviewed as part of the project because limited resources could cause important

logs or packet evidence to be removed too quickly. To reduce that risk, the project focused on safer controls such as retention tuning, reducing unnecessary Zeek logging, and avoiding aggressive filtering too early. Overall, risk management in this project meant identifying where problems could happen, reducing the chance of harm to systems or data, and keeping the work controlled enough to produce useful results in a live environment.

Chapter 2: Background and Related Work

Overview

This chapter will explain the background and related work for my project. Since my project is focused on network intrusion detection and investigation using Security Onion, the most important background areas are passive monitoring, mirrored traffic collection, intrusion detection sensors, packet capture, and the use of tools like Suricata and Zeek to turn traffic into alerts and network metadata. These technologies are widely used in the security field because they help organizations detect suspicious activity, review network behavior, and support investigations with stronger evidence (Bhatia & Almukhaini, 2024; Chinnasamy et al., 2025).

This chapter also references previous work from both published research and other student projects that relate to network security monitoring and investigation workflows (Davis, 2024; Headland, 2024). Some projects focused on network hardening, secure network design, wireless threat detection and/or security alerts which are all relevant to my project because they display different ways to improve visibility and reduce risk in real environments. My project is somewhat different because it is centered on the deployment and validation of a Security Onion sensor in a live enterprise environment, with the goal of turning monitored traffic into alerts, logs, and packet evidence that can be repeated and supported in the investigation process.

The purpose of this chapter is to show where my project fits within the larger field of cybersecurity and to explain how it builds upon existing ideas while also solving a practical problem in a more useful way. It provides the foundation for the next chapter by showing what technologies are already being used and what other work has been done while also showing where my project adds its own practical contributions.

Framework and Threat Model Alignment

This project aligns with NIST Cybersecurity Framework 2.0 as well as MITRE ATT&CK in different parts. From a standards perspective, the work aligns well with the NIST Cybersecurity Framework 2.0, especially the Detect and Respond functions. The sensor deployment, mirrored traffic collection, alert validation, and log review all support detection by improving the organization's ability to identify suspicious network activity. The investigation workflow described in this project also supports response because it helps move from raw traffic and alerts to a clearer explanation of what happened during a suspicious event, which makes follow-up investigation and incident handling more practical (National Institute of Standards and Technology, 2024).

Additionally, this project also aligns with the MITRE ATT&CK framework at the tactic level. The main north-south monitoring focus was intended to improve visibility into behaviors related to command and control and possible exfiltration, while the planned future expansion into east-west monitoring would improve visibility into lateral movement inside the environment. Using this ATT&CK-based view helps show that the project was not only a technical deployment, but also a practical monitoring design aimed at detecting behaviors that are relevant to real enterprise attacks (MITRE ATT&CK, n.d.).

Existing systems or approaches

There are several systems and approaches that are already used in the field of network intrusion detection and network security monitoring. One of the most common approaches is passive monitoring, where traffic is mirrored from a switch SPAN, TAP, or similar monitoring path to a dedicated sensor instead of placing the sensor inline with production. This method is widely used because it allows organizations to inspect packets, generate alerts, and review

network behavior without directly interfering with normal communication. Security Onion supports this kind of monitoring model and combines multiple tools into one platform for intrusion detection, threat hunting, log management, and packet analysis (Security Onion Solutions, LLC, n.d.).

Another common approach is to combine signature-based detection with protocol metadata and packet evidence instead of relying on only one source of information. This means using tools such as Suricata for alert generation, Zeek for network and protocol logging, and keeping packet capture for deeper review when needed. According to Bhatia and Almukhaini (2024), Security Onion can be an effective open-source solution for cyber-attack detection because it brings together multiple network security monitoring functions in one environment and can successfully detect a significant portion of tested attack activity. Even though their study was based on a proof-of-concept lab rather than a live enterprise deployment, their work is still very useful as a background example because it shows how Security Onion has already been evaluated as a detection platform.

Existing approaches in related student work also show that network security projects often depend on layered monitoring, hardening, and validation rather than on a single tool. For example, Headland (2024) combined a hardened network design with a centralized security monitoring platform and alerting workflow, showing that security tools become more useful when they support both detection and a clear response process. In a different way, Davis (2024) focused on network hardening and penetration testing to evaluate whether defensive configurations could hold up against realistic threats. These projects are relevant because they show that security solutions are most effective when they move beyond simple deployment and are tested in a way that helps explain what happened and why.

Overall, the existing solutions and projects in this area show that the field already has strong tools for monitoring and detection, but their success depends on correct deployment, useful traffic visibility, and the ability to connect alerts, logs, and packet evidence into one investigation process.

Limitations of current systems

Current intrusion detection and network monitoring solutions are useful, but they still contain significant gaps. One of these limitations is that many systems are strong in one area but weaker in another. For example, signature-based tools are effective at detecting known threats, but they can miss new or unusual behavior if it does not match an existing rule. Chinnasamy et al. (2025) explain that traditional security tools often struggle with delayed threat detection, high alert volume, scalability problems, and difficulty handling modern enterprise traffic at a large scale. In the same way, Bakraouy et al. (2025) show that relying on only one detection method is not enough, which is why many newer systems combine packet inspection, behavioral logging, and centralized analysis.

Detection alone does not provide enough context for investigation. While a system may generate alerts, it might not provide useful network metadata, searchable logs, and packet evidence, which makes it difficult to understand what happened. Bakraouy et al. (2025) point out that stronger results come from combining Suricata for signature-based inspection, Zeek for behavioral and protocol logging, and Elastic Stack for centralized visibility. This is important because many existing systems and solutions leave gaps between seeing an alert and being able to explain the full event clearly. In related student work, Headland (2024) also showed that security tools become more useful when they support a larger response and monitoring process rather than acting as isolated tools.

Another existing gap is between controlled test environments and real enterprise deployment. Bhatia and Almukhaini (2024) found that Security Onion could detect a significant portion of simulated attacks in a proof-of-concept lab. That result is useful; however, a lab is much more controlled than a live enterprise environment. There are a lot of limitations that go into real enterprise environments: privacy restrictions, limited access, approval requirements, resource limits, and sensitive traffic that cannot always be tested or documented freely. Because of that, there is still a need for work that shows not only that these tools can detect threats, but also how they can be prepared, validated, and used practically in a real monitored environment.

My project addresses part of this gap by focusing on the deployment and validation side of the problem. I am not only treating Security Onion as a tool that should work in theory but focusing on making the sensor usable in practice through interface separation, mirrored traffic monitoring, alert and log validation, and a repeatable investigation process using Suricata, Zeek, and packet evidence. In that way, the project contributes a more practical solution to the common problem of having security data available but not having an easy or repeatable way of investigating it.

Conclusion

This chapter reviewed the main background and related work connected to my project. It showed that network intrusion detection already uses several important approaches such as passive monitoring, mirrored traffic collection, signature-based detection, protocol logging, and packet capture. It also showed that tools such as Security Onion, Suricata, and Zeek are already being used to improve network visibility and support threat detection. The chapter also explained that current systems still have important gaps, especially when moving from controlled testing into real enterprise environments where deployment, validation, privacy, and

investigation workflow become more difficult. Previous research and related student projects helped show both the value of these technologies and the limits that still exist in practice (Bakraouy et al., 2025; Bhatia & Almukhaini, 2024; Davis, 2024; Headland, 2024).

My project is built on the idea of focusing on a more practical part of the problem. Instead of only looking at detection in theory or in a lab, it focuses more on preparing and validating a Security Onion sensor in a real enterprise environment and using it as part of a repeatable investigation process. This creates a direct connection between the related work discussed in this chapter and the work completed in my own project.

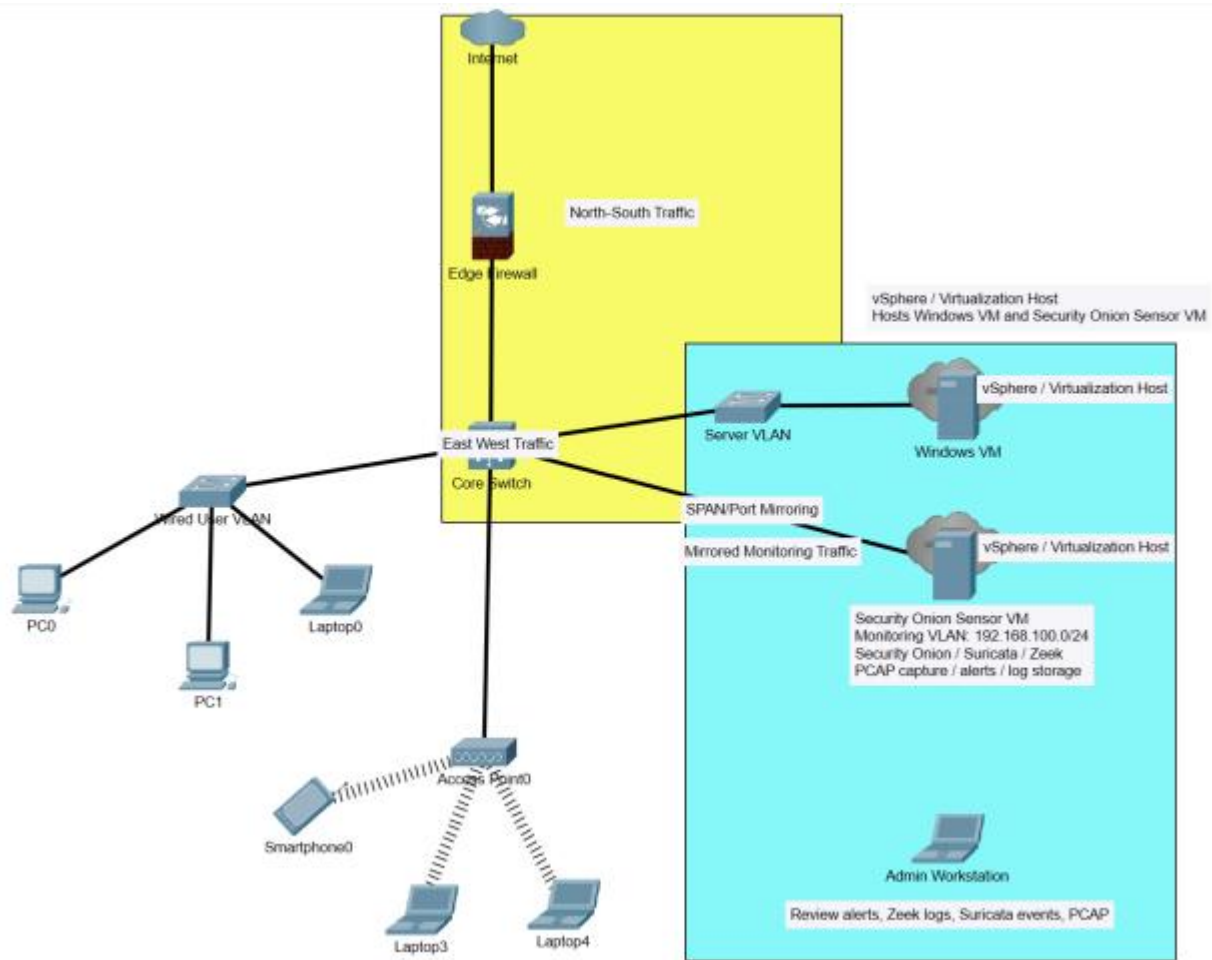
The next chapter moves from background into the actual project itself. It explains the problem in more detail, describes the solution I used, and presents the design, setup, methods, and configuration choices that were used to complete the project.

Chapter 3: Problem statement and system design

3.1 Overview and Problem Statement

Network visibility is a major part of cybersecurity because modern organizations depend heavily on their networks to support daily operations, communication, access to services, and the movement of important data. If suspicious traffic cannot be seen clearly, suspicious activity may be missed due to alerts not having enough context. This becomes even more difficult in an enterprise environment, where access is restricted, infrastructure is sensitive, and system changes must be planned carefully to avoid affecting production systems. Simply having a security tool installed is not enough. In such an environment, the tool must be configured in a way that gives useful visibility and supports investigation.

The main problem that arose in my working environment was that Security Onion was operating in an Elastic-only configuration. That setup could collect endpoint logs and show alerts in the interface, but it did not provide the full network sensor capabilities needed for Suricata, Zeek, and full packet capture. As a result, it was not possible to fully monitor mirrored network traffic or use packet-based evidence as part of a complete investigation workflow. Without a proper SPAN deployment, there was a gap between seeing security data in general and having the specific network-level evidence needed to validate suspicious activity in a practical and repeatable way.



(Figure 1, High-level network diagram)

The project problem was therefore not just to use Security Onion, but to prepare, validate, and deploy the sensor side of a passive monitoring workflow in a real enterprise environment. That meant making sure the virtual sensor was properly configured, that management and monitoring traffic were separated, that approved mirrored traffic could reach the sensor without interrupting production systems, and that the sensor could generate useful alerts, logs, and packet evidence for investigation. This also came with a lot of challenges such as limited access, privacy restrictions, enterprise documentation limits, storage concerns, and the challenge of proving that the monitoring workflow was functioning the way it was intended to function.

Another part of the challenge was testing the system before the SPAN deployment completion. Part of the work involved preparing the sensor in advance, confirming service health, identifying the correct monitoring interface, validating that traffic capture was possible, configuring settings such as HOME_NET, and troubleshooting how Suricata and Zeek data appeared in the system. This testing phase was important because, in a real enterprise environment, there is always a risk that a configuration change or deployment step could disrupt normal operations or negatively affect production systems. For that reason, it was necessary to validate the design, confirm the planned workflow, and test the configuration in advance before applying it more broadly in the environment.

This problem is both practical and technical. Without an effective sensor-based monitoring process, the organization has reduced visibility across its network, fewer opportunities to investigate activity at the packet level, and less ability to link alerts to clear supporting evidence. As a result, detecting threats and analyzing incidents becomes more difficult, especially when there is a need to explain exactly what happened during suspicious activity. This project addresses that issue by focusing on the practical steps required to move from a limited Elastic-only view toward a broader network monitoring and investigation workflow using a virtual Security Onion sensor.

3.2 Methodologies

The methodology for this project was based on a staged implementation process rather than a single deployment step. Since the project took place in a real enterprise environment, the work had to be done carefully, with preparation and validation completed before the final monitoring workflow could be trusted. Before deploying the sensor, I first had to understand the limitations of the existing platform, validate its health and configuration, confirm its ability to capture mirrored

traffic, and use that foundation to support a repeatable investigation workflow. This phased method was necessary because in a live enterprise setting, changes could not be made without testing, and each part of the monitoring process must be confirmed before moving to the next step.

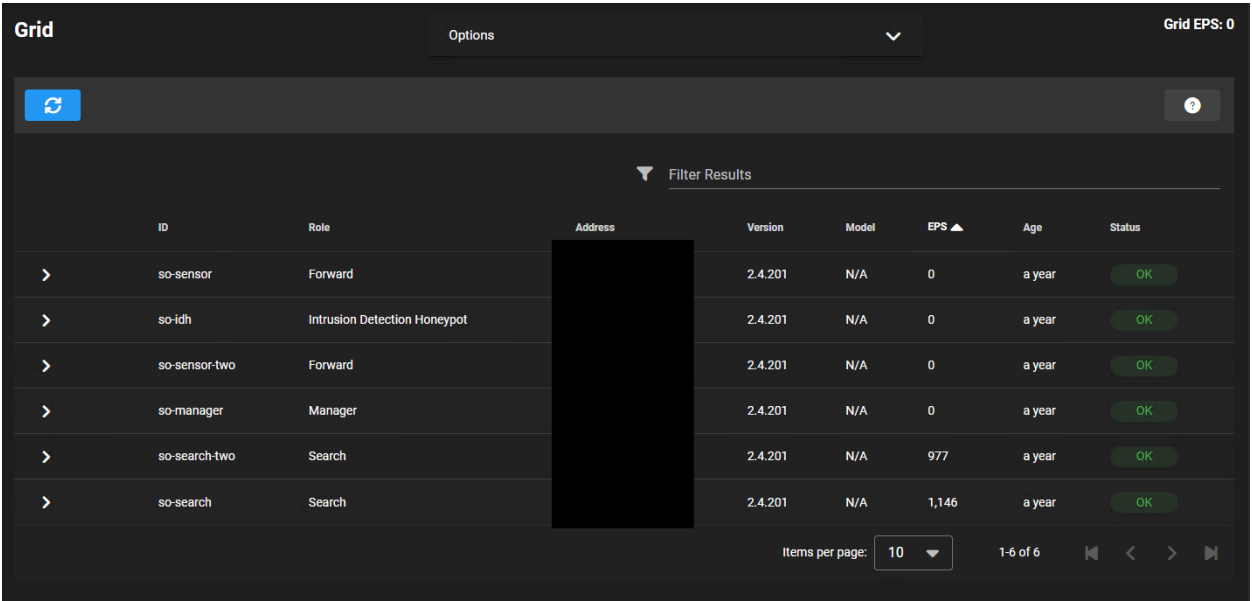
3.2.1 Initial Planning and Design Approach

The first stage of the project was planning how to build a sensor-based monitoring approach. At the beginning, the environment was operating in the Elastic-only setup which did not provide the full network sensor capabilities needed for Suricata, Zeek, and packet capture. Due to this, the project had to begin with the design of a more complete passive monitoring workflow. That design centered on using a dedicated Security Onion sensor, a mirrored traffic source, a separate monitoring interface, and controlled retention settings so that network evidence could be collected without affecting production traffic.

The planning also included deciding what traffic would be most valuable to monitor. The decision was made to emphasize north-south visibility because internet-facing traffic is often the most useful for identifying initial access, command and control, and data exfiltration patterns. At the same time, the project left room for selected east-west visibility, which will be executed in the future. During this stage, the project also considered practical limits such as storage usage, retention, packet capture load, and the need to avoid aggressive filtering too early in the process. This was important because, in a real enterprise environment, there is always a risk that a configuration change or deployment step could disrupt normal operations, negatively affect production systems, or miss important alerts due to incorrect configurations. For that reason, it was necessary to validate the design, confirm the planned workflow, and test the configuration in advance before applying it more broadly in the environment.

3.2.2 Sensor Preparation and Platform Validation

The next stage after planning was the preparation of the virtual sensor. The Security Onion sensor was deployed in a vSphere-based virtual environment and configured so that management and monitoring functions remained separate. This stage focused on confirming that the sensor was online, healthy, and properly recognized by the Security Onion platform. The Grid view was used to verify that the sensor node existed and was communicating with the rest of the platform, while status checks confirmed that the main services were available and stable. This preparation stage was necessary because the monitoring workflow could not be trusted until the platform side of the sensor was shown to be operational. Additional platform and service validation evidence is included in Appendix A.



The screenshot shows the 'Grid' view of the Security Onion platform. At the top, there's a header with 'Grid', 'Options', and 'Grid EPS: 0'. Below the header is a table with columns: ID, Role, Address, Version, Model, EPS, Age, and Status. The table lists six nodes: so-sensor, so-idh, so-sensor-two, so-manager, so-search-two, and so-search. The Address column for all nodes is redacted with a black box. The Status column shows 'OK' for all nodes. At the bottom, there's a pagination bar showing 'Items per page: 10' and '1-6 of 6'.

ID	Role	Address	Version	Model	EPS	Age	Status
> so-sensor	Forward		2.4.201	N/A	0	a year	OK
> so-idh	Intrusion Detection Honeypot		2.4.201	N/A	0	a year	OK
> so-sensor-two	Forward		2.4.201	N/A	0	a year	OK
> so-manager	Manager		2.4.201	N/A	0	a year	OK
> so-search-two	Search		2.4.201	N/A	977	a year	OK
> so-search	Search		2.4.201	N/A	1,146	a year	OK

(Figure 2, Security Onion Grid view showing sensor and platform node status)

Once the node was visible in the platform, direct validation was performed from the sensor itself. The sensor was accessed through SSH, and system health was checked using the command **so-status**. This step was used to confirm that core monitoring components such as Suricata, Zeek,

and packet capture were active before deeper testing began. Verifying this directly from the sensor was important because platform visibility alone does not always prove that the individual services needed for monitoring and evidence collection are functioning correctly.

```
[soadmin@so-sensor-two ~]$ sudo so-status
[sudo] password for soadmin:

Security Onion Status
```

Container	Status	Details
so-sensoroni	running	Up 55 minutes
so-steno	running	Up 55 minutes
so-strelka-backend	running	Up 54 minutes
so-strelka-coordinator	running	Up 55 minutes
so-strelka-filestream	running	Up 54 minutes
so-strelka-frontend	running	Up 55 minutes
so-strelka-gatekeeper	running	Up 55 minutes
so-strelka-manager	running	Up 55 minutes
so-suricata	running	Up 55 minutes
so-telemetry	running	Up 40 minutes
so-zeek	running	Up 55 minutes (healthy)

```

✓ This onion is ready to make your adversaries cry!
[soadmin@so-sensor-two ~]$
```

(Figure 3, **so-status** output confirming active Security Onion monitoring services)

3.2.3 Interface Identification and Monitoring Readiness

After confirming the sensor was healthy, the next stage was identifying and validating the correct interface roles. The goal was to determine which interface handled management traffic and which one was reserved for monitoring. The step was crucial because the project depended on passive inspection, which means that the monitoring side of the sensor had to receive the copied traffic without becoming part of the normal production path. Interface checks were performed using system commands to identify the management interface, the monitoring interface, and the physical network adapter associated with the monitoring path. Additional validation was done by checking bond details, physical link state, and whether the monitoring side of the sensor was receiving traffic. Supporting interface and bond validation evidence is included in Appendix B.

```
[soadmin@so-sensor-two ~]$ cat /proc/net/bonding/bond0
Ethernet Channel Bonding Driver:

Bonding Mode: load balancing (round-robin)
MII Status: up
MII Polling Interval (ms): 100
Up Delay (ms): 0
Down Delay (ms): 0
Peer Notification Delay (ms): 0

Slave Interface: ens224
MII Status: up
Speed: 10000 Mbps
Duplex: full
Link Failure Count: 0
Permanent HW addr:
Slave queue ID: 0
```

(Figure 4, Bond interface details confirming the monitoring interface configuration)

As part of the early validation procedure, traffic capture was used. Commands such as **tcpdump** were used on the monitoring side to confirm that packets could be collected and observed. At this stage, the goal was not to prove that the full SPAN deployment was complete, but to separate sensor-side readiness from switch-side configuration issues. This stage helped to determine whether the sensor could see traffic at all before blaming any later visibility issues on the monitoring tools themselves. This stage of validation was important because it reduced uncertainty and helped narrow troubleshooting later in the project. Additional packet capture validation evidence is included in **Appendix C**.

```

[soadmin@so-sensor-two ~]$ sudo tcpdump -nni ens224
dropped privs to tcpdump
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on ens224, link-type EN10MB (Ethernet), snapshot length 262144 bytes
17:04:17.758114 ARP, Request who-has tell length 46
17:04:17.886643 IP .255 215
17:04:18.048005 ARP, Request who-has ell length 46
17:04:18.052767 ARP, Request who-has tell length 46
17:04:18.079043 ARP, Request who-has ell ngth 46
17:04:18.299207 ARP, Request who-has ell length 46
17:04:18.320610 ARP, Request who-has ell ngth 46
17:04:18.757905 ARP, Request who-has tell length 46
17:04:19.052898 ARP, Request who-has tell length 46
17:04:19.071904 ARP, Request who-has ell ngth 46
17:04:19.220645 ARP, Request who-has ell length 46
17:04:19.266987 ARP, Request who-has ell length 46
17:04:19.297782 ARP, Request who-has ell ngth 46
17:04:19.741339 IP BOOT rom length 300
17:04:20.052802 ARP, Request who-has tell length 46
17:04:20.177431 ARP, Request who-has ell ngth 46
^C
16 packets captured
16 packets received by filter
0 packets dropped by kernel

```

(Figure 5-1, **tcpdump** capture on the physical monitoring interface)

```

[soadmin@so-sensor-two ~]$ sudo tcpdump -nni bond0
[sudo] password for soadmin:
dropped privs to tcpdump
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on bond0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
17:00:48.425457 ARP, ..... length 46
17:00:48.895055 ARP, ..... length 46
17:00:50.020090 IP 1 UDP, length 50
17:00:50.144994 IP 1 UDP, length 50
17:00:50.375194 ARP, 168.100.50, length 46
17:00:50.635481 IP 1 : UDP, length 50
17:00:50.679712 IP 1 : UDP, length 50
17:00:50.784917 IP 1 UDP, length 50
17:00:50.924900 ARP, .168.100.26, length 46
17:00:50.941151 IP 1 UDP, length 50
17:00:51.040353 ARP, length 46
17:00:51.382094 ARP, , length 46
17:00:51.391058 ARP, length 46
17:00:51.409013 IP 1 : UDP, length 50
17:00:51.471964 IP 1 : UDP, length 50

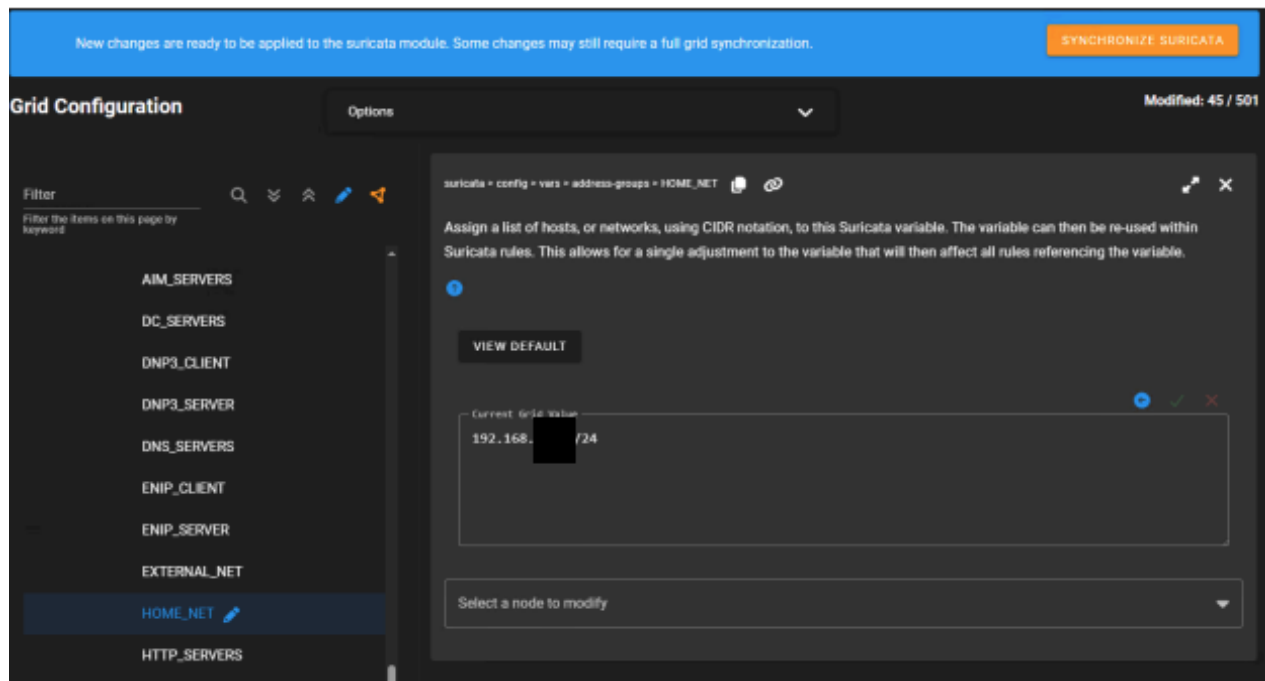
```

(Figure 5-2, **tcpdump** capture on the bonded monitoring interface)

3.2.4 Configuration and Internal Tuning

After identifying the monitoring path, the next stage was configuring Security Onion so the sensor would interpret traffic correctly. One important part of this stage was setting `HOME_NET` to match the monitored subnet used for testing. This was done to make the alerts and logs easier to understand and to reduce confusion during analysis. Service restarts and synchronization steps were then used to apply the changes to the sensor. In addition to `HOME_NET`, this stage also

included reviewing time and time zone settings so the timestamps shown in alerts and logs would be accurate and consistent during investigation. Additional configuration and time synchronization evidence is included in **Appendix D**.



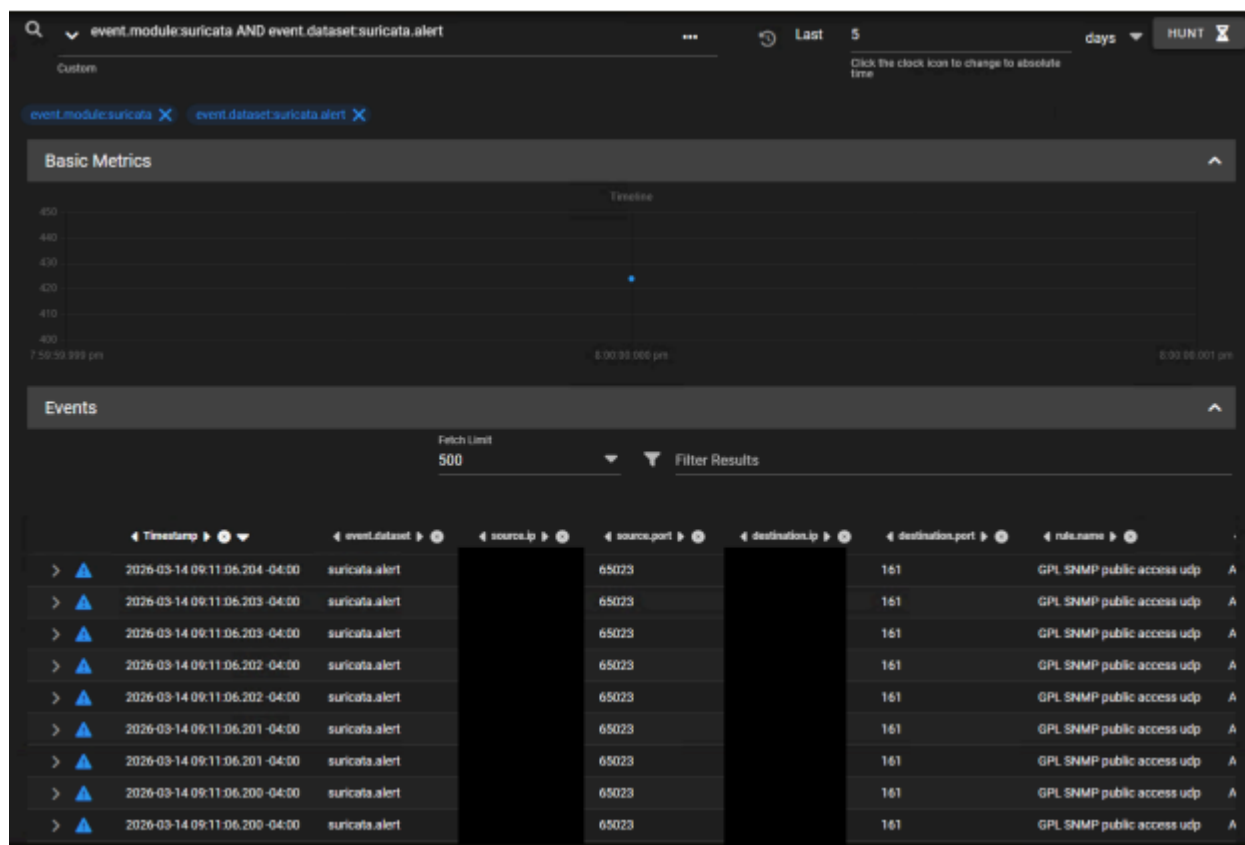
(Figure 6, HOME_NET configuration for the monitored subnet in Security Onion)

Another part of this stage was resource and retention planning. Packet capture and log storage can consume a significant amount of space, because of that part of this stage also included reviewing PCAP retention, reducing unnecessary Zeek logging, and choosing lower-risk tuning options before using more aggressive filtering. Instead of relying on BPF filters and risking the loss of important traffic, I took a safer approach by controlling data volume through retention changes, selective Zeek logging, and alert tuning plans. This helped lower the chance of accidentally filtering out traffic that might later be important during an investigation. Additional configuration and tuning evidence are included in **Appendix D**.

3.2.5 Alert, Log, and Workflow Validation

The next step in the process was validating and troubleshooting the actual monitoring outputs after the sensor configuration had been prepared. This stage focused on confirming that Security Onion was not only running, but also producing useful evidence through Suricata alerts, Zeek metadata, and packet evidence. This process involved checking where Suricata wrote its EVE output, verifying log locations, confirming that rotating EVE files were being created, and understanding how those events appeared in the Hunt interface. This troubleshooting was an important part of the methodology because it showed that a working sensor still requires careful validation before its outputs can be trusted to be used for future investigations. Additional Hunt and backend alert validation evidence is included in **Appendix E**.

The project also tested how alerts and logs could be viewed and interpreted inside Security Onion. Hunt was used as the main interface for filtering and separating different data sources, especially when distinguishing between endpoint data and Suricata alert data. Zeek connection and DNS visibility were also reviewed as a part of this stage because the project goal was not only to receive alerts, but to connect those alerts to supporting context. This proved that a useful network monitoring process depends on more than one tool or one alert view. It also depends on the ability to move from traffic visibility to alert validation to evidence review in a consistent way. Additional Hunt and alert validation evidence is included in **Appendix E**.



(Figure 7, Hunt view showing indexed Suricata alert events in Security Onion)

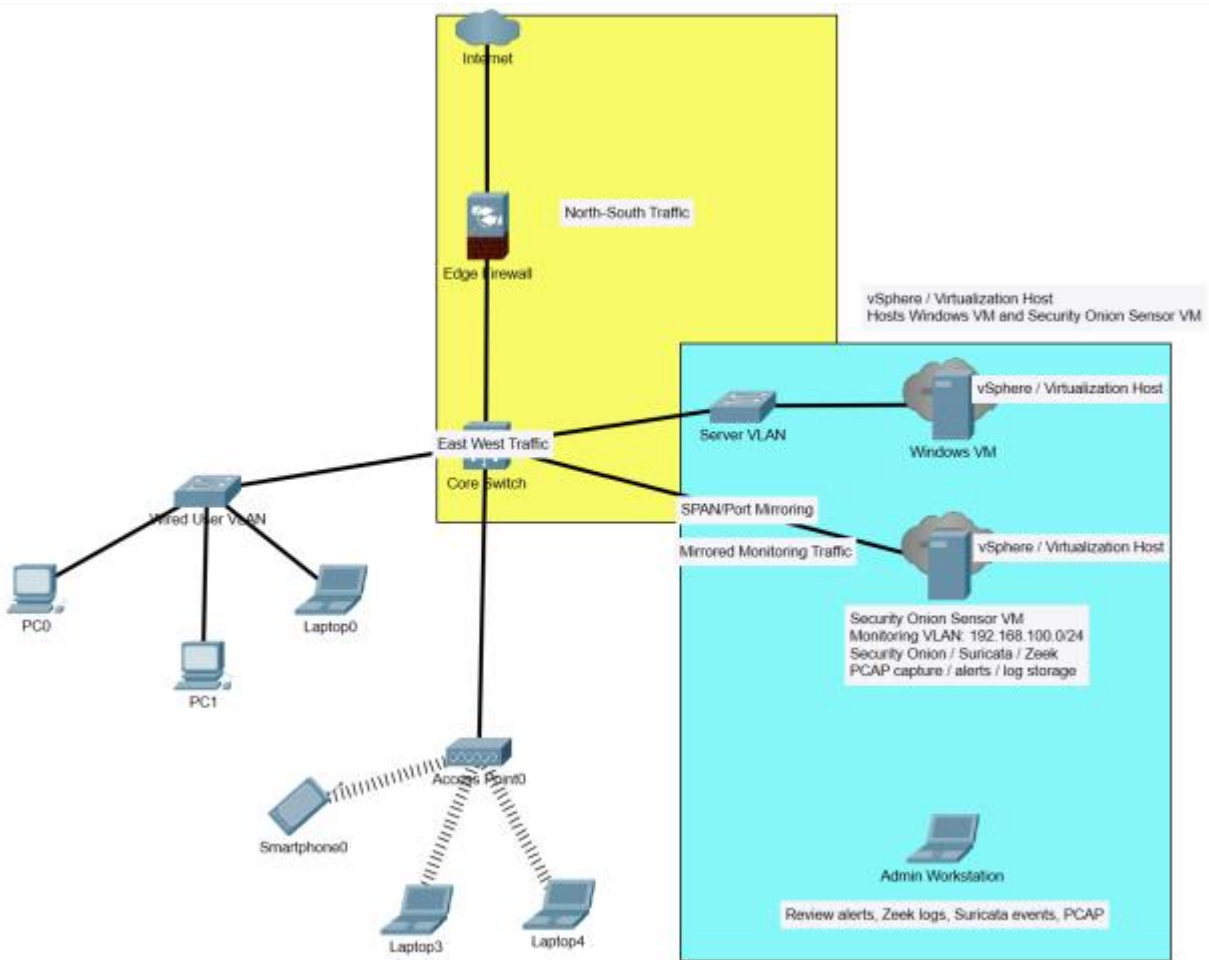
Overall, the goal for this project was not just a single installation step but a staged enterprise workflow. It began with identifying the limits of the existing environment and then moved through sensor preparation, platform health validation, interface identification, monitoring-path testing, configuration tuning, and alert and log verification. These steps were necessary because the project was based on real infrastructure rather than a simple lab simulation, and each part of the process had to be tested before the full monitoring and investigation workflow could be considered reliable.

3.3 System design

3.3.1 Overall Architecture

The system design for this project was based on a passive network monitoring model using a virtual Security Onion sensor in a live enterprise environment. The main goal of the design was to improve network visibility while also keeping the production traffic from any interruptions or changes in the normal communication flow of the environment. Instead of placing the sensor inline, the system was designed so that approved traffic would be copied through a mirrored monitoring path and sent to the sensor for inspection. This allowed the setup to collect security data in a controlled way while reducing the risk of affecting live business operations.

At a high level, the design included the internet edge, firewall, core switching layer, internal network segments, a mirrored traffic path, the Security Onion sensor, and an administration workstation used to review outputs. The Security Onion sensor was positioned so that it could receive copied traffic from the network rather than act as a forwarding device itself. Once the copied traffic reached the sensor, the platform could process it using Suricata, Zeek, and packet capture functions. This design was chosen because it supports monitoring and investigation while keeping the security platform separate from the main production path.



(Figure 1, high-level network design.)

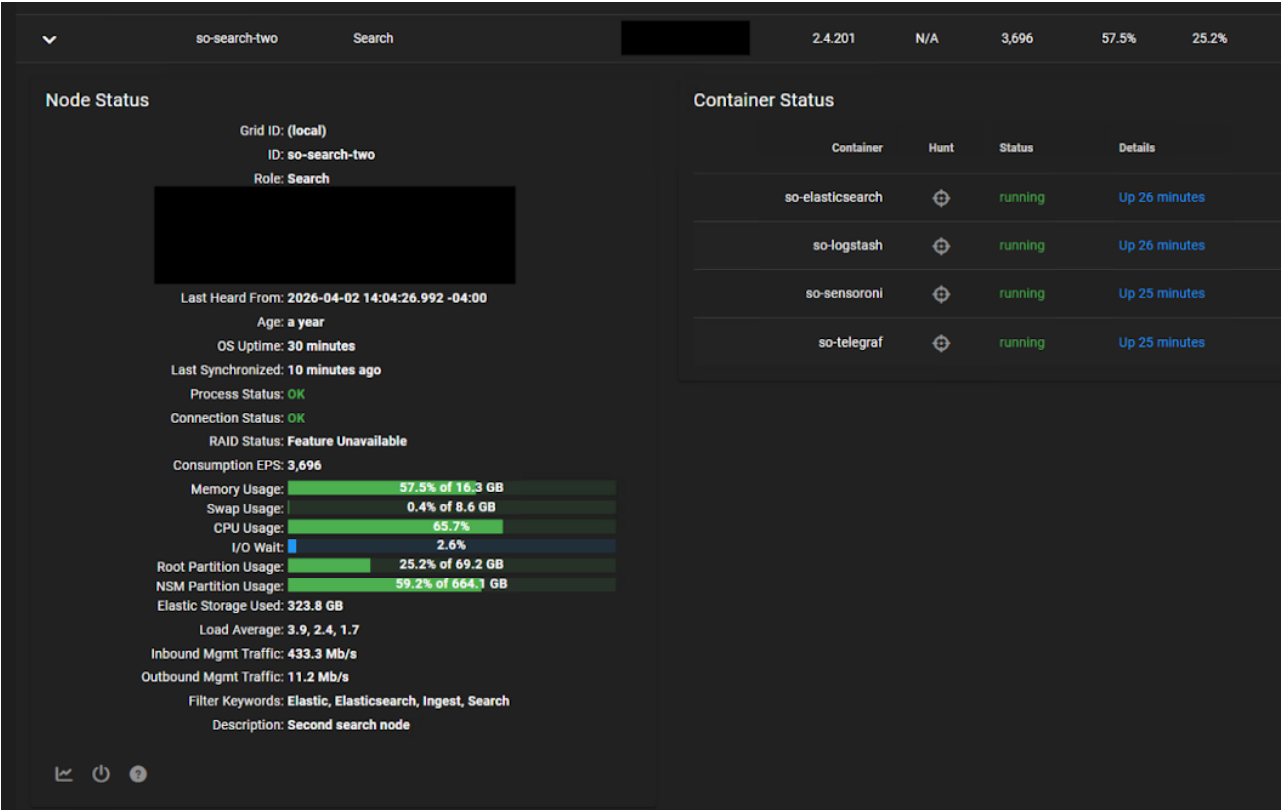
This design made the architecture suitable for enterprise use because it supported visibility without creating a point of failure in the main production traffic path. The design also left room for both north-south visibility and selected east-west visibility in the further expansion. Overall, the architecture was designed to balance security visibility, operational safety, and realistic deployment restrictions.

3.3.2 Sensor Role and Interface Design

My project focused on the sensor side of Security Onion, which included deploying, preparing, and validating a virtual sensor that could receive monitored traffic and convert that traffic into useful investigation data, rather than trying to redesign or rebuild the entire platform.

This also means that my project was not intended to demonstrate a full manager deployment from scratch. Instead, it focused on creating a sensor workflow that could operate inside the existing enterprise environment and extend it with network monitoring capabilities.

The sensor was designed with separate interfaces for management and monitoring. This separation was important because administration traffic and monitored network traffic needed to remain independent of each other. Keeping those two interfaces separate was crucial because they performed two different roles. The management side of the sensor was used for access, updates, configuration, and communication with the rest of the Security Onion platform, while the monitoring side was reserved only for receiving copied traffic, making it suitable for passive monitoring. This also reduced the chance of mixing administration traffic with monitored packet data and made the monitoring path easier to validate.



(Figure 8. Security Onion sensor-two and container status after sensor configuration changes)

3.3.3 Sensor Deployment and Monitoring Path

As explained earlier, one side of the sensor was used for normal administration, while the other side was reserved only for monitored traffic. To support this design, after confirming that the sensor was visible in Security Onion and that the main services were running correctly, I connected to the sensor through SSH and checked the platform from the command line. This helped me confirm that the system was healthy and that I was ready for the next steps. The next step was identifying and documenting the correct interfaces for both management and monitoring sides. Then I checked the bond configuration to confirm how the monitoring interface was mapped inside (as shown earlier in Figure 4).

After the sensor was prepared and the interface roles were confirmed, the next step was creating the mirrored traffic path. The Security Onion deployment stayed fully virtual inside the vSphere environment. No physical rewiring or direct hands-on work with network equipment was performed. Instead, the existing virtual infrastructure was used so that copied traffic could be delivered to the Security Onion virtual machine through a dedicated monitoring path, while the management side remained separate for normal administrative operations.

For this design, I kept one virtual network adapter for management and used the second adapter as a sniffing interface for monitored traffic only. This separation is necessary because the monitoring side was not intended to function as a normal user or management connection. Its only purpose was to receive copied packets for inspection by the Security Onion monitoring tools.

Once the sensor side was ready, the next step was to work on preparing the virtual networking path that would carry mirrored traffic. To prevent ordinary management traffic from being mixed with copied packets, the monitoring adapter had to be attached to the correct distributed port group. I also reviewed the port group security settings that could affect whether

mirrored traffic would reach the sensor. These settings included promiscuous mode, MAC address behavior, and forged transmit handling, since those configuration areas can determine whether copied frames are accepted or blocked before Security Onion ever sees them.

After that, I defined the traffic that would be mirrored for the deployment. The main goal of the deployment was to mirror north-south traffic related to the firewall, while also leaving room for future east-west expansion, so the project could move beyond monitoring only the default infrastructure VLAN used during early testing. The copied packet stream was then delivered to the Security Onion virtual machine through the monitoring path so the sensor could inspect the traffic passively without changing its placement in the environment.

3.3.4 SPAN Validation and Investigation Workflow

After arranging the mirrored traffic path, the next step was validating that the deployment was working correctly. The first check was the monitoring interface itself rather than waiting for the alerts to appear. Packet capture was used on the monitoring side to confirm that mirrored packets were actively reaching the sensor. This was important to confirm packet visibility before expecting Security Onion to generate usable monitoring data.

Once packet visibility was confirmed, I moved to platform-level validation. I checked whether Security Onion was processing the mirrored traffic correctly and whether usable monitoring output was being generated from that packet stream. This included reviewing platform status and confirming that traffic seen on the monitoring side was being turned into meaningful monitoring data through Suricata alerts and connection metadata.

During this process, I also checked other configuration areas that could affect whether mirrored traffic was interpreted correctly. These included VLAN-related behavior, the scope of HOME_NET, monitoring-side security settings, and whether the expected monitoring rules were

enabled. Checking these areas was necessary because simply receiving packets did not guarantee that Security Onion would generate useful alerts.

First, I confirmed that the monitoring side of the sensor could see the mirrored traffic directly. Then, I confirmed that Security Onion was converting that traffic into alerts, logs, and connection data that would be usable to support future investigations. This created the repeatable workflow used throughout the project: confirm packet visibility, confirm monitoring output, review alert details, examine supporting metadata, and summarize the event in a short evidence-based explanation.

3.3.5 Repeatable Investigation Workflow

The next step is to review the alert or event details, including the time, source, destination, protocol, and rule information. After that, related metadata such as connection or DNS records can be reviewed for any additional information. Packet evidence can then be used to support deeper analysis. The final step is to summarize the event into a short, evidence-based explanation. A practical example of this workflow is presented later in Chapter 4 using one of the alerts collected during the project.

Chapter 4: Results and Analysis

4.1 Outcomes overview

The main outcome of this project was the successful preparation, validation, and use of a Security Onion sensor workflow in a real enterprise environment. By the end of the project completion, the deployment had moved beyond an Elastic-only setup and was able to support network monitoring through a virtual sensor, mirrored traffic, and passive analysis. This improved network visibility in the environment and made it possible to review monitored traffic through Security Onion using Suricata alerts, Zeek metadata, and packet-level evidence.

The final result was able to create a usable monitoring process that included sensor preparation, interface validation, mirrored traffic delivery, configuration tuning, and investigation support. The project also produced practical evidence that the monitoring workflow was functioning. This included confirmation that mirrored traffic was reaching the monitoring side of the sensor, that Security Onion was processing the traffic, and that usable event data could be reviewed inside the platform. These outcomes are important because they show that the project addressed the original problem of limited network visibility and improved the earlier Elastic-only configuration.

4.2 Experimental results

The results of the project were developed in stages, beginning with validation of the sensor itself, and ending with confirmation that monitored traffic was being turned into useful security data. At the beginning, the sensor was confirmed to be active and healthy in the Security Onion platform, and direct status checks showed that important monitoring services were running.

The next result was confirmation that traffic could be observed on the monitoring side of the sensor. The first direct evidence of the functioning monitoring path came from packet capture checks, which showed that the monitoring interface was receiving traffic. Once packet visibility was confirmed, the project moved to validating whether Security Onion could process that monitored traffic and produce usable output.

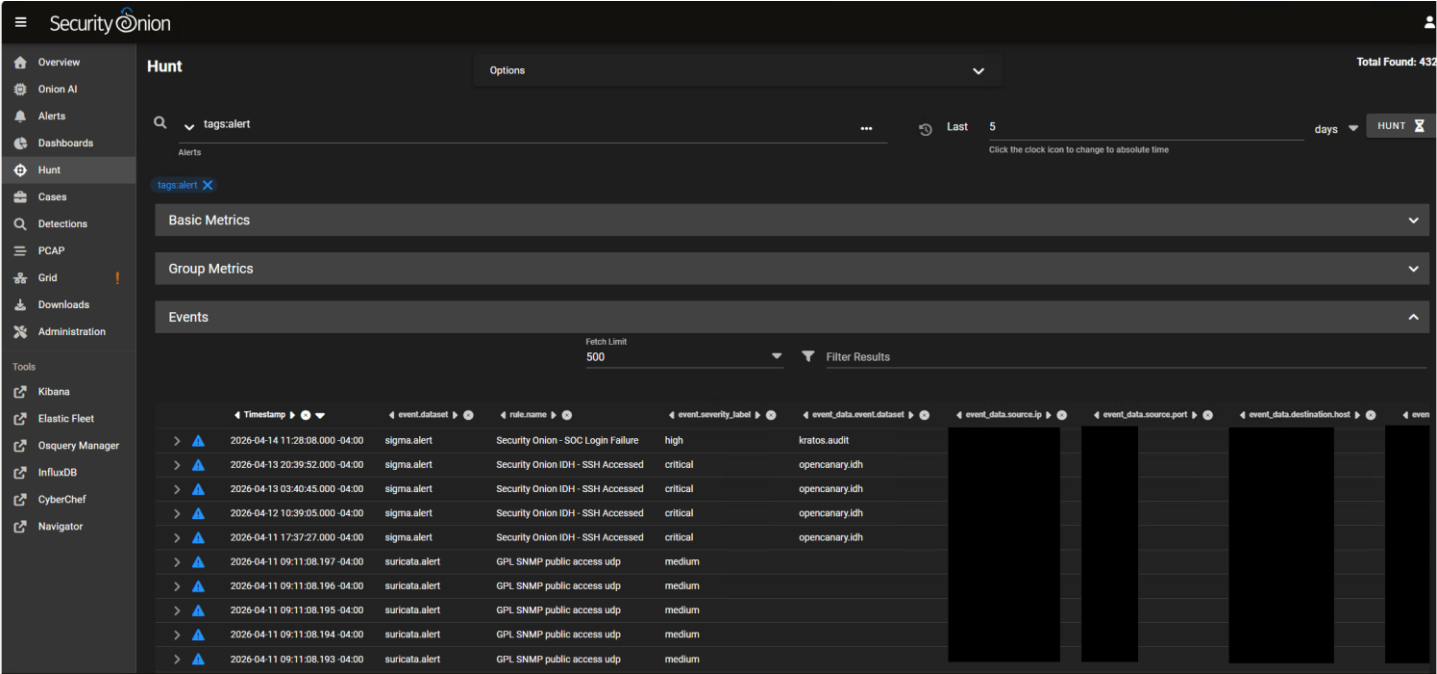
Additional results came from configuration and monitoring validation. The monitored subnet was defined through HOME_NET so that alerts and logs could be interpreted more accurately. Time settings and synchronization were also checked to make sure event timing was reliable. After this, Suricata and Zeek outputs were reviewed in the platform, especially through Hunt, to confirm that monitored data was searchable and usable for investigation.

4.2.1 Example Investigation of a Collected Alert

One example collected during the project was a Suricata alert labeled **GPL SNMP public access udp**. This alert was identified in the Security Onion Hunt view by filtering alert-tagged events. As shown in Figure 9, the event list included multiple records of the same alert type, making it a useful example for demonstrating how the investigation workflow could be applied to a real monitored event.

After selecting one of the records, the detailed event view confirmed that the alert was a **suricata.alert** event and showed additional investigation fields such as timestamp, severity, destination information, DNS query content, and the log file path. The event was labeled with medium severity and was clearly tied to Suricata processing rather than only a general platform message. As shown in Figure 10, the detailed fields helped confirm that the alert was real, searchable, and suitable for review inside the platform.

This example was useful because it showed that the sensor was not only receiving monitored traffic but also converting that traffic into usable alert data that could be reviewed in Hunt. Although longer-term supporting context was limited by the configured retention period, the alert itself still demonstrated the intended workflow: the event could be identified in Hunt, opened in detail, and interpreted using the key fields provided by Security Onion. That made it a practical example of how the deployed sensor supported investigation rather than only raw traffic collection. Supporting Hunt and event detail screenshots are also provided in Appendix E.



(Figure 9, Hunt results showing the selected Suricata SNMP alert)

Another important finding was that successful deployment depended on more than simply enabling monitoring tools. The results showed that interface separation, mirrored traffic delivery, subnet definition, and output validation all affected whether the final workflow was usable. This is important because it shows that real enterprise monitoring depends on both system configuration and infrastructure design, not just software installation.

The project also showed that the most useful results came from combining different types of evidence. Alerts provided the starting point, but logs and packet visibility added to the context needed to understand what happened more clearly. This made the workflow more practical for investigation and reporting than relying on alerts alone.

4.4 Comparison with existing solutions

The findings from this project can be compared with existing research showing that Security Onion and related open-source monitoring tools can provide useful intrusion detection and investigation support when they are deployed correctly. One important part of this project is that this work was not based on a complete step-by-step guide that already existed for this exact type of enterprise deployment. From what I was able to find, there was no complete online documentation that matched the project I completed, so the design, validation, and troubleshooting process had to be worked out through my own research, testing, and implementation. However, there have been published studies and related projects that were useful for comparison, such as Bhatia and Almukhaini (2024), who found that Security Onion was effective in detecting a significant portion of simulated cyberattacks in a test-lab environment. My project supports that general idea, but in a different way. Instead of focusing on a proof-of-concept lab with simulated attacks, my project focused on preparing and validating the sensor workflow in a real enterprise environment. That makes the contribution

different because it emphasizes deployment reality, traffic visibility, and investigation workflow rather than only attack simulation results.

Bakraouy et al. (2025) similarly showed that combining Suricata, Zeek, and the Elastic Stack provides stronger visibility than relying on only one detection method, which supports the layered workflow used in my project. This is also consistent with Chinnasamy et al. (2025), who noted that modern network security tools face challenges related to scalability, alert handling, and effective traffic analysis in enterprise environments. Taken together, these comparisons show that my project aligns with existing research but contributes a more practical example of how a Security Onion sensor can be prepared, validated, and used in a real enterprise monitoring environment. This framework-based view also supports the project's relevance to recognized best practices in cybersecurity monitoring and incident analysis (National Institute of Standards and Technology, 2024; MITRE ATT&CK, n.d.).

Conclusions and Future Work

Summary of Achievements

This project successfully turned an existing Security Onion setup from a limited Elastic-only environment into a more complete network monitoring workflow. The most important achievement was preparing, validating, and using a virtual Security Onion sensor that could receive mirrored traffic, process it through Suricata and Zeek, and support investigation using alerts, logs, and packet evidence. Another major achievement was building a workflow that was not based on a ready-made guide but instead had to be developed through my own planning, research, testing, troubleshooting, and validation in a real enterprise environment. Overall, the project showed that a properly prepared sensor can improve network visibility and make investigations more practical and evidence-based.

Limitations of the Project

This project had several limitations beginning with access and control. Because the work was completed in a real enterprise environment, I could not make unrestricted infrastructure changes on my own and had to work within approval processes, supervision, and access boundaries. Another limitation involved documentation and screenshot use. Many interfaces, configurations, and monitoring views contained sensitive internal details, which meant that some screenshots had to be heavily redacted and others could not be included at all. This also limited how much detailed configuration information could be shown in the final report.

Another limitation was that I could not find any prior guide or closely similar project for this type of work in a real enterprise setting. That made the work more difficult because much of the deployment logic, troubleshooting process, and validation approach had to be figured out

independently. In addition, storage and retention limits affected how long logs and packet evidence could be kept, which reduced the amount of historical data available at times. Finally, because the project was carried out in a live environment, testing had to remain controlled and non-disruptive, which limited how much active experimentation could be performed compared to a full lab setup.

Possible Future Improvements or Extensions

A clear future improvement would be expanding traffic coverage further, especially by adding more east-west monitoring so that the sensor can observe more internal lateral movement and communication between systems inside the environment. Another useful improvement would be refining long-term retention so that alerts, logs, and packet evidence could be stored more consistently without creating storage pressure. The project could also be extended by further tuning alerts, improving investigation dashboards and filtering, and adding more documented real-world examples of how alerts can be followed from detection to evidence-based explanation. Over time, these improvements would make the monitoring workflow stronger, easier to maintain, and more useful for practical security operations.

References

Bakraouy, Z., Abbass, W., Boughanja, M., Flissat, M., Baina, A., & Bellafkih, M. (2025). Hybrid NIDS architecture leveraging Suricata, Zeek and the ELK stack. In 2025 International Conference on Intelligent Systems: Theories and Applications (SITA). IEEE.

<https://doi.org/10.1109/SITA67914.2025.11273397>

Bhatia, G., & Almukhaini, G. (2024). Enhancing cyber attack detection: An analysis of Security Onion for small and midsize enterprise. In 2024 1st International Conference on Innovative Engineering Sciences and Technological Research (ICIESTR). IEEE.

<https://doi.org/10.1109/ICIESTR60916.2024.10798271>

Chinnasamy, P., SivaKrishnaiah, C., Anjali, T., Kambalapelly, A., Rao, P. V., & Degala, D. P. (2025). Managing network security in IT sector using the Suricata. In Proceedings of the 3rd International Conference on Self Sustainable Artificial Intelligence Systems (ICSSAS-2025). IEEE. <https://doi.org/10.1109/ICSSAS66150.2025.11080990>

Davis, M. (2024). Constructing and probing a fortified network. Williams Honors College, Honors Research Projects, 1791.

https://ideaexchange.uakron.edu/honors_research_projects/1791

Headland, C. A. (2024). Mitigating cyber espionage: A network security strategy using notifications. Williams Honors College, Honors Research Projects, 1790.

https://ideaexchange.uakron.edu/honors_research_projects/1790

MITRE ATT&CK. (n.d.). ATT&CK®. MITRE.

<https://attack.mitre.org>

National Institute of Standards and Technology. (2024). The NIST cybersecurity framework (CSF) 2.0. U.S. Department of Commerce. <https://www.nist.gov/publications/nist-cybersecurity-framework-csf-20>

Security Onion Solutions, LLC. (n.d.). Introduction. Security Onion Documentation (Version 2.4). Retrieved January 21, 2026, from <https://docs.securityonion.net/en/2.4/introduction.html>

Appendices

Appendix A. Security Onion Service and Node Validation

This appendix contains supporting evidence that the Security Onion platform and sensor node were active, healthy, and running the required monitoring services.

Options

Appendix A-1. Grid view showing the main nodes and their status

Node Status

Grid ID: (local)

ID: so-sensor-two

Role: Forward

Address: [REDACTED]

Version: 2.4.201

Model: N/A

Date Created: 2024-10-30 10:53:59.000 -04:00

Earliest PCAP: 2026-01-07 09:26:28.000 -05:00

Last Heard From: 2026-04-14 12:28:07.523 -04:00

Age: a year

OS Uptime: 3 hours

Last Synchronized: 13 minutes ago

Process Status: OK

Connection Status: OK

RAID Status: Feature Unavailable

Memory Usage: 55.1% of 12.1 GB

Swap Usage: 0.0% of 8.6 GB

CPU Usage: 2.5%

I/O Wait: 0.0%

Capture Loss: 0.0%

Zeek Loss: 0.0%

Suricata Loss: 0.0%

Suricata Rules: 48852 loaded

Last Suricata Reload: 2026-04-14 09:20:05.472 -04:00

Stenographer Loss: 0.0%

Root Partition Usage: 27.7% of 69.2 GB

NSM Partition Usage: 25.8% of 132.6 GB

PCAP Retention: 2.1 days

Load Average: 0.0, 0.0, 0.0

Inbound Monitor Traffic: 0.0 Mb/s

Dropped Monitor Traffic: 0.0 Mb/s

Inbound Mgmt Traffic: 0.0 Mb/s

Outbound Mgmt Traffic: 0.0 Mb/s

Filter Keywords: Forward, Sensor, Sensoroni, Stenographer

Description: Second sensor node for security onion



Appendix A-2. Detailed node status view showing node health.

Container Status			
Container	Hunt	Status	Details
so-sensoroni		running	Up 3 hours
so-steno		running	Up 3 hours
so-strelka-backend		running	Up 3 hours
so-strelka-coordinator		running	Up 3 hours
so-strelka-filestream		running	Up 3 hours
so-strelka-frontend		running	Up 3 hours
so-strelka-gatekeeper		running	Up 3 hours
so-strelka-manager		running	Up 3 hours
so-suricata		running	Up 3 hours
so-telegraf		running	Up 3 hours
so-zeek		running	Up 3 hours (healthy)

Appendix A-3. Detailed node status view showing running containers.

```
[soadmin@so-sensor-two ~]$ sudo so-status
[sudo] password for soadmin:

Security Onion Status
```

Container	Status	Details
so-sensoroni	running	Up 55 minutes
so-steno	running	Up 55 minutes
so-strelka-backend	running	Up 54 minutes
so-strelka-coordinator	running	Up 55 minutes
so-strelka-filestream	running	Up 54 minutes
so-strelka-frontend	running	Up 55 minutes
so-strelka-gatekeeper	running	Up 55 minutes
so-strelka-manager	running	Up 55 minutes
so-suricata	running	Up 55 minutes
so-telegraf	running	Up 40 minutes
so-zeek	running	Up 55 minutes (healthy)

✓ This onion is ready to make your adversaries cry!

Appendix A-4. so-status output showing Suricata, Zeek, and related services running.

Appendix B. Interface and Monitoring Path Validation

This appendix provides supporting evidence for interface identification, bond mapping, and the separation of management and monitoring functions in the virtual sensor design.

```
[soadmin@so-sensor-two ~]$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
2: ens192: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state UP group default qlen 1000
    link/ether 
    altnam enp11s0
    inet 
        brd 
        scope global noprefixroute ens192
    2
        valid_lft forever preferred_lft forever
    inet6 
        64 scope link noprefixroute
        valid_lft forever preferred_lft forever
3: ens224: <BROADCAST,NOARP,PROMISC,SLAVE,UP,LOWER_UP> mtu 9000 qdisc mq master bond0 state UP group default qlen 1000
    link/ether 
    altnam enp19s0
4: bond0: <BROADCAST,MULTICAST,PROMISC,MASTER,UP,LOWER_UP> mtu 9000 qdisc noqueue state UP group default qlen 1000
    link/ether 
    brd
```

Appendix B-1. Interface listing showing management and monitoring paths

```
[soadmin@so-sensor-two ~]$ cat /proc/net/bonding/bond0
Ethernet Channel Bonding Driver: v5.15.0-317.197.5.1.el9uek.x86_64

Bonding Mode: load balancing (round-robin)
MII Status: up
MII Polling Interval (ms): 100
Up Delay (ms): 0
Down Delay (ms): 0
Peer Notification Delay (ms): 0

Slave Interface: ens224
MII Status: up
Speed: 10000 Mbps
Duplex: full
Link Failure Count: 0
Permanent HW addr:
Slave queue ID: 0
```

**Appendix B-2. Bond interface details confirming the monitoring interface
configuration**

Appendix C. Traffic Capture Validation

This appendix contains packet capture evidence used to confirm that mirrored traffic was reaching the monitoring side of the sensor.

```
[soadmin@so-sensor-two ~]$ sudo tcpdump -nni ens224
[sudo] password for soadmin:
dropped privs to tcpdump
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on ens224, link-type EN10MB (Ethernet), snapshot length 262144 bytes
16:45:39.466840 ARP, Request who-has tell : , length 46
16:45:39.554790 IP UDP, length 50
16:45:39.570352 IP UDP, length 50
16:45:39.650347 ARP, Request who-has tell length
16:45:39.736057 ARP, Request who-has tell length 4
16:45:39.744783 ARP, Request who-has tell length
16:45:39.979601 ARP, Request who-has tell length 46
16:45:40.106568 ARP, Request who-has tell length 46
16:45:40.338311 ARP, Request who-has tell length
16:45:40.784217 ARP, Request who-has tell length 4
16:45:40.979091 ARP, Request who-has tell , length
16:45:41.338231 IP UDP, length 50
16:45:41.369630 IP UDP, length 50
16:45:41.603500 ARP, Request who-has tell length
```

Appendix C-1. tcpdump capture on the physical monitoring interface

```

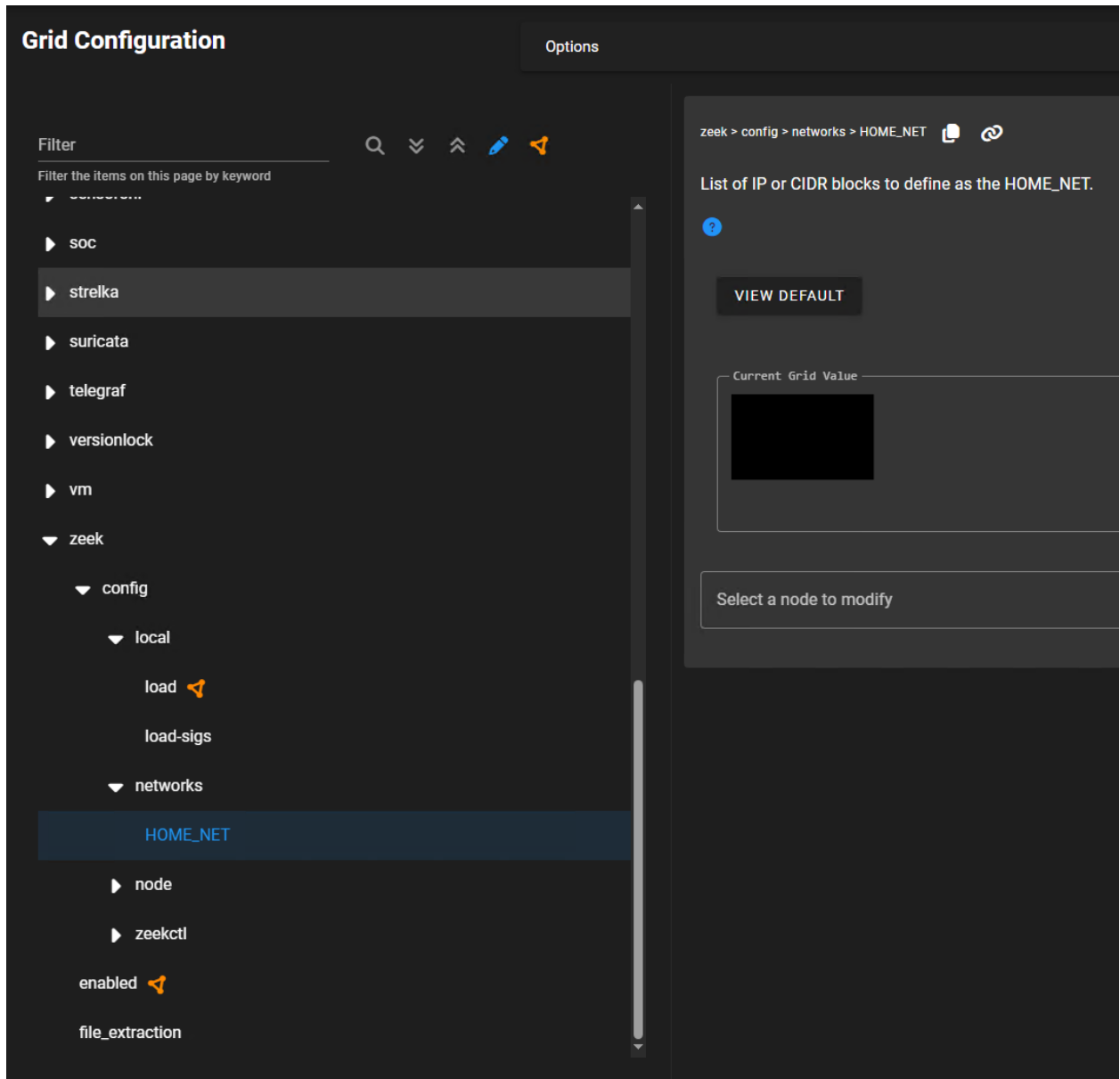
[soadmin@so-sensor-two ~]$ sudo tcpdump -nni bond0
dropped privs to tcpdump
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on bond0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
6:47:46.885846 ARP, Request who-has tell , length 46
6:47:46.935263 ARP, Request who-has tell , length 46
6:47:47.142595 IP > : UDP, length 50
6:47:47.383423 ARP, Request who-has tell , length 46
6:47:47.417692 ARP, Request who-has , length 46
6:47:47.590948 IP JDP, length 50
6:47:47.892611 IP UDP, length 50
6:47:48.104551 ARP, Request who-has tell , length 46
6:47:48.121316 ARP, Request who-has tell , length 46
6:47:48.355283 IP UDP, length 50
6:47:48.401086 ARP, Request who-has tell length 46
6:47:48.792654 ARP, Request who-has tell length 46
6:47:49.114110 ARP, Request who-has tell length 46
6:47:49.150907 ARP, Request who-has tell length 46
6:47:49.214179 ARP, Request who-has tell length 46
6:47:49.425077 ARP, Request who-has tell length 46
6:47:49.435157 ARP, Request who-has tell length 46
6:47:49.735603 ARP, Request who-has 2 tell , length 46
6:47:49.901188 ARP, Request who-has tell length 46
6:47:50.354512 ARP, Request who-has tell length 46

```

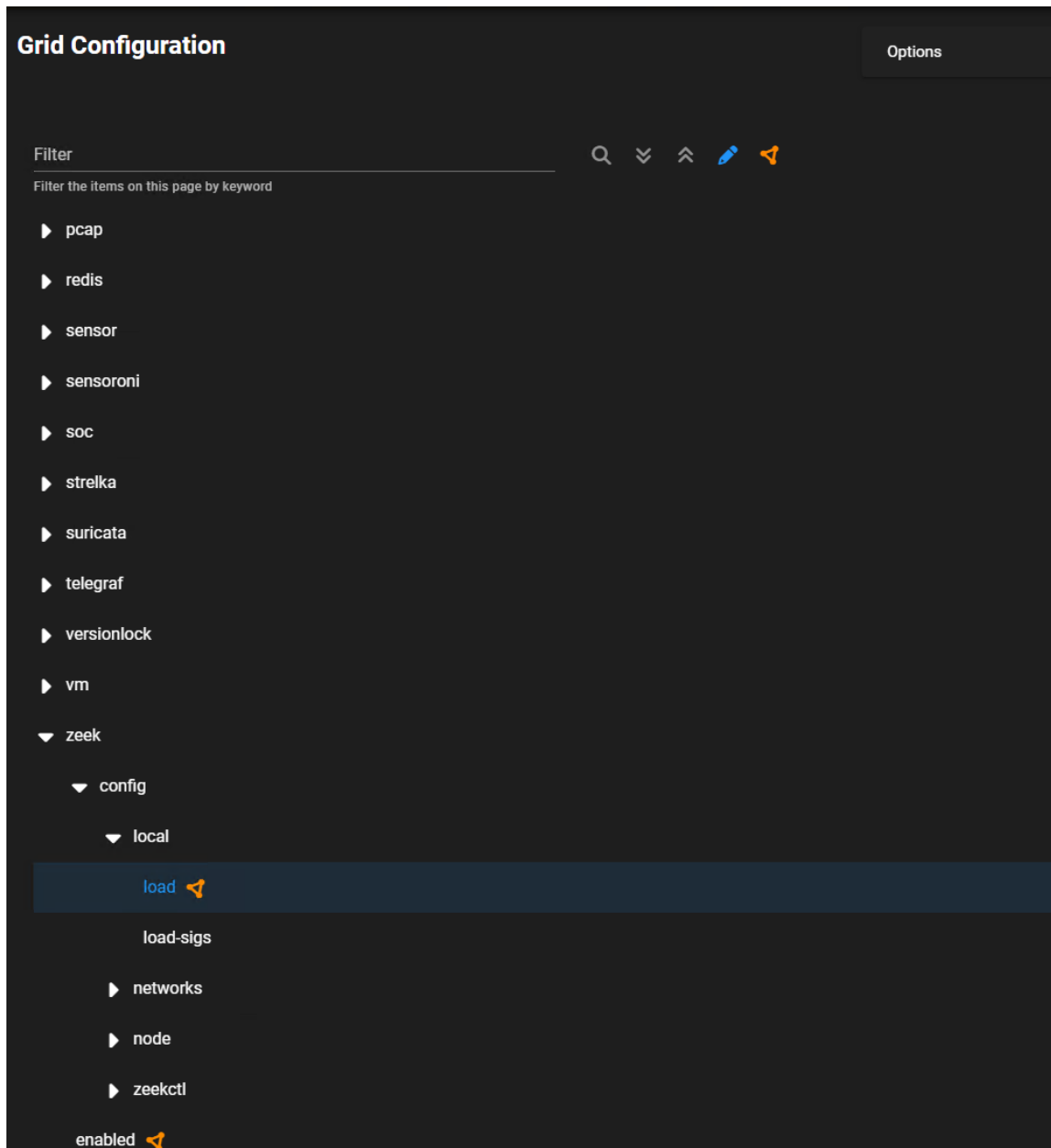
Appendix C-2. tcpdump capture on the bonded monitoring interface

Appendix D. Configuration and Tuning Evidence

This appendix provides supporting evidence for key configuration and tuning decisions, including monitored subnet definition and related detection settings.



Appendix D-1. HOME_NET configuration for the monitored subnet



Appendix D-2. Configuration view showing tuning-related settings



Appendix D-3.1. Enabled rules or plugin-related configuration evidence

```
so-search-two_searchnode
```

```
intel
```

```
securityonion/bpfconf
```

```
securityonion/community-id-extended
```

```
oui-logging
```

```
icsnpp-modbus
```

```
icsnpp-dnp3
```

```
icsnpp-bacnet
```

```
icsnpp-ethercat
```

```
icsnpp-enip
```

```
icsnpp-opcua-binary
```

```
icsnpp-bsap
```

```
icsnpp-s7comm
```

```
zeek-plugin-tds
```

```
zeek-plugin-profinet
```

```
zeek-spicy-wireguard
```

```
zeek-spicy-stun
```

```
http2
```

```
zeek-spicy-ipsec
```

```
zeek-spicy-openvpn
```

Appendix D-3.2. Enabled rules or plugin-related configuration evidence

```
[soadmin@so-manager ~]$ sudo timedatectl set-timezone America/New_York && timedatectl | egrep "Local time|Time zone|System clock synchronized|NTP service"
[sudo] password for soadmin:
      Local time: Tue 2026-03-17 10:35:31 EDT
      Time zone: America/New_York (EDT, -0400)
System clock synchronized: yes
      NTP service: active
[soadmin@so-manager ~]$
```

**Appendix D-4. timedatectl output confirming synchronized time settings on the
Security Onion manager**

```
[soadmin@so-sensor-two ~]$ sudo timedatectl set-timezone America/New_York && timedatectl | egrep "Local time|Time zone|System clock synchronized|NTP service"
[sudo] password for soadmin:
      Local time: Tue 2026-03-17 10:35:49 EDT
      Time zone: America/New_York (EDT, -0400)
System clock synchronized: yes
      NTP service: active
[soadmin@so-sensor-two ~]$
```

**Appendix D-5. timedatectl output confirming synchronized time settings on the
Security Onion sensor**

Appendix E. Hunt, Alert, and Investigation Evidence

This appendix includes supporting Hunt and alert review evidence used to show that the deployed sensor produced searchable monitoring data and supported investigation workflow validation.

The screenshot displays the 'Alerts' section of a Security Onion interface. The left sidebar contains navigation links: Overview, Onion AI, Alerts (selected), Dashboards, Hunt, Cases, Detections, PCAP, Grid, Downloads, Administration, Tools, Kibana, Elastic Fleet, and Osquery Manager. The main panel shows a table of alerts grouped by name and module. The table has columns for Count, rule.name, event.module, event.severity_label, and rule.uuid. The data is sorted by rule.name. The top five rows are:

Count	rule.name	event.module	event.severity_label	rule.uuid
527	Security Onion IDH - SSH Accessed	sigma	critical	fcf94210-f188-4
325	Security Onion IDH - SSH Login Attempt	sigma	critical	6b033764-7a14
3,230	Security Onion - Grid Node Login Failure (SSH)	sigma	high	923421c7-9b1e
2	Security Onion - SOC Login Failure	sigma	high	bf86ef21-41e6-
424	GPL SNMP public access udp	suricata	medium	2101411

The interface also includes a search bar, a 'Group By Name, Module' dropdown, a 'Fetch Limit' of 500, a 'Filter Results' button, and a 'Total Found: 4,508' indicator. The bottom of the table shows 'Items per page: 50' and '1-5 of 5'.

Appendix E-1. Alerts view showing grouped alert activity in Security Onion

Hunt

Options

▼

event.category:network AND tags:alert | groupby rule.category rule.gid rule.uid rule.name

event.category:network X top:alert X

1005 Alerts

...

Last 5

Click the clock icon to change to absolute time

Total

days ▼

Basic Metrics

Group Metrics

Events

500

Fetch Limit

▼

Filter Results

⏪ Timestamp ⏩	⏪ event.timestamp ⏩	⏪ source.ip ⏩	⏪ source.port ⏩	⏪ destination.ip ⏩	⏪ destination.port ⏩	⏪ rule.name ⏩	⏪ rule.category ⏩	⏪ event.severity_label ⏩	⏪ log_level ⏩	⏪ network.community_id ⏩
> ▲ 2025-04-11 09:11:07.641 -04:00	suricata.alert					GPL SNMP public access udp	Attempted Information Leak	medium		
> ▲ 2025-04-11 09:11:07.670 -04:00	suricata.alert					GPL SNMP public access udp	Attempted Information Leak	medium		
> ▲ 2025-04-11 09:11:07.759 -04:00	suricata.alert					GPL SNMP public access udp	Attempted Information Leak	medium		
> ▲ 2025-04-11 09:11:07.823 -04:00	suricata.alert					GPL SNMP public access udp	Attempted Information Leak	medium		
> ▲ 2025-04-11 09:11:07.872 -04:00	suricata.alert					GPL SNMP public access udp	Attempted Information Leak	medium		
> ▲ 2025-04-11 09:11:08.171 -04:00	suricata.alert					GPL SNMP public access udp	Attempted Information Leak	medium		
> ▲ 2025-04-11 09:11:08.186 -04:00	suricata.alert					GPL SNMP public access udp	Attempted Information Leak	medium		
> ▲ 2025-04-11 09:11:07.491 -04:00	suricata.alert					GPL SNMP public access udp	Attempted Information Leak	medium		
> ▲ 2025-04-11 09:11:07.563 -04:00	suricata.alert					GPL SNMP public access udp	Attempted Information Leak	medium		
> ▲ 2025-04-11 09:11:07.575 -04:00	suricata.alert					GPL SNMP public access udp	Attempted Information Leak	medium		

Items per page: 10 1-10 of 424

Appendix E-2. Hunt view showing indexed Suricata alert events

Alerts

tags alert X

Options

Total Found: 437

🔍 tags:alert

...

Last 5

days

HUNT

Click the clock icon to change to absolute time

Events

Group Metrics

Basic Metrics

🔍 Filtering

Filter Results

Timestamp	event:dataset	ml:username	event:severity_label	event:data.event:dataset	event:data.source:ip	event:data.source:port	event:data.destination:host	event:
2026-04-14 11:28:08.000 -04:00	sigma.alert	Security Onion - SOC Login Failure	high	knitos.audit				
2026-04-13 20:39:32.000 -04:00	sigma.alert	Security Onion IDH - SSH Accessed	critical	opencanary/ridh				
2026-04-13 03:40:45.000 -04:00	sigma.alert	Security Onion IDH - SSH Accessed	critical	opencanary/ridh				
2026-04-12 10:39:05.000 -04:00	sigma.alert	Security Onion IDH - SSH Accessed	critical	opencanary/ridh				
2026-04-11 17:37:27.000 -04:00	sigma.alert	Security Onion IDH - SSH Accessed	critical	opencanary/ridh				
2026-04-11 09:11:08.197 -04:00	suricata.alert	GPI SNMP public access udp	medium					
2026-04-11 09:11:08.196 -04:00	suricata.alert	GPI SNMP public access udp	medium					
2026-04-11 09:11:08.195 -04:00	suricata.alert	GPI SNMP public access udp	medium					
2026-04-11 09:11:08.194 -04:00	suricata.alert	GPI SNMP public access udp	medium					
2026-04-11 09:11:08.193 -04:00	suricata.alert	GPI SNMP public access udp	medium					

▼	🚨	2026-04-11 09:11:08.197 -04:00	suricata.alert	GPL SNMP public access udp	medi
📁	📄	@timestamp	2026-04-11T13:11:08.197Z		
📁	📄	@version	1		
📁	📄	data_stream.dataset	suricata		
📁	📄	data_stream.namespace	so		
📁	📄	data_stream.type	logs		
📁	📄	destination.ip	[REDACTED]		
📁	📄	destination.port	[REDACTED]		
📁	📄	dns.query_name	public		
📁	📄	ecs.version	8.0.0		
📁	📄	elastic_agent.id	[REDACTED]		
📁	📄	elastic_agent.snapshot	false		
📁	📄	elastic_agent.version	8.18.8		
📁	📄	event.category	network		
📁	📄	event.dataset	suricata.alert		
📁	📄	event.ingested	2026-04-11T13:11:09.645Z		
📁	📄	event.module	suricata		
📁	📄	event.severity	2		
📁	📄	event.severity_label	medium		
📁	📄	input.type	log		
📁	📄	log.file.path	/nsm/suricata/eve-2026-04-11-13:10.json		
📁	📄	log.id.uid	945832143894006		
📁	📄	log.offset	307603		
📁	📄	message	{ "timestamp": "2026-04-11T13:11:08.197704+0000", "flow_id": 945832143894006, "signature": "GPL SNMP public access udp", "category": "Attempted Information Disclosure", "content": "public", "fast_pattern": true, "reference": "bugtraq,2112; CVE-2019-11358", "generated_by": "Proofpoint_Nexus", "updated_at": "2019-10-08", "app_proto": "snmp", "type_name": "EN10MB" }		
📁	📄	metadata.beat	filebeat		
📁	📄	metadata.input.beats.host.ip	[REDACTED]		

Appendix E-4. Detailed event view for the selected Suricata SNMP alert

```
[soadmin@so-sensor-two ~]$ sudo zcat /nsm/suricata/eve-2026-03-07-14:16.json.gz | grep -o '"event_type":"'[^"]*' | head -n 30
"event_type":"alert"
"event_type":"alert"
"event_type":"alert"
"event_type":"alert"
"event_type":"alert"
"event_type":"alert"
"event_type":"alert"
"event_type":"alert"
"event_type":"alert"
"event_type":"alert"
"event_type":"alert"
"event_type":"alert"
"event_type":"alert"
"event_type":"alert"
"event_type":"alert"
"event_type":"alert"
"event_type":"alert"
"event_type":"alert"
"event_type":"alert"
"event_type":"alert"
"event_type":"alert"
"event_type":"alert"
"event_type":"alert"
"event_type":"alert"
"event_type":"alert"
"event_type":"alert"
"event_type":"alert"
"event_type":"alert"
"event_type":"alert"
```

Appendix E-5. Command output confirming Suricata EVE alert events in the compressed log file

```
[soadmin@so-sensor-two ~]$ sudo zcat /nsm/suricata/eve-2026-03-07-14:16.json.gz | grep -o '"signature":"'[^"]*' | head -n 15
"signature":"GPL SNMP public access udp"
"signature":"GPL SNMP public access udp"
"signature":"GPL SNMP public access udp"
"signature":"GPL SNMP public access udp"
"signature":"GPL SNMP public access udp"
"signature":"GPL SNMP public access udp"
"signature":"GPL SNMP public access udp"
"signature":"GPL SNMP public access udp"
"signature":"GPL SNMP public access udp"
"signature":"GPL SNMP public access udp"
"signature":"GPL SNMP public access udp"
"signature":"GPL SNMP public access udp"
"signature":"GPL SNMP public access udp"
"signature":"GPL SNMP public access udp"
"signature":"GPL SNMP public access udp"
[soadmin@so-sensor-two ~]$
```

Appendix E-6. Command output showing GPL SNMP public access udp signatures in the Suricata EVE log